# Report – Project Computer Graphics and Visual Computing (OpenGL)

Course: Computer Graphics & Visual Computing
Academic Year: 2025–2026
Group Members: Denis Topallaj & Maxim Peeters

---

## Implemented Features

### 1. Curve (Denis)

The scene contains two closed 3D Bézier curves, each composed of four cubic Bézier segments that together form a smooth circular loop. The control points are generated using the well-known approximation constant `k = 0.552284749831 * r` (the "magic number" `4*(sqrt(2)-1)/3`, see source in `src/main.cpp`) so that four cubic segments faithfully approximate a full circle. The two loops differ in radius and vertical amplitude:

- **Loop 1:** radius = 10, height oscillation = +/-2
- **Loop 2:** radius = 7, height oscillation = +/-5

Each curve is built from 13 control points spanning 4 segments. Curve evaluation uses the standard cubic Bernstein formula, and the tangent is computed analytically as the derivative of that polynomial. Both curves are visualised as a white polyline rendered with 400 sample points via `BezierCurveRenderer`.

### 2. Animation – Constant Speed via Arc Length (Denis)

To guarantee that the train moves at a *constant speed* regardless of the local curvature of the curve, a **look-up table (LUT)** based on arc length is built for each Bézier curve (`BezierCurve::buildLUT`, 400 samples). Each entry stores a `(t, arcLength)` pair. During the render loop, a global distance accumulator is incremented by `deltaTime x 5.0` units/second, and `getTFromDistance()` linearly interpolates inside the LUT to recover the corresponding curve parameter `t`. This means the position is always sampled at the arc-length-correct parameter, not at a uniformly-spaced `t`, which would produce uneven speed.

### 3. Models and Textures (Denis)

3D geometry is represented as a rectangular box mesh (a train carriage shape defined by 36 vertices with positions, normals, and UV coordinates) loaded through the custom `Mesh` class. A PNG texture (`src/resources/train-carriage.png`) is loaded via the `Texture` class using `stb_image` and applied through the material diffuse sampler in the fragment shader. The same mesh geometry is reused for the railroad tie blocks and the clickable button object.

### 4. Visualisation – Railroad Track Bending with the Curve (Denis)

The active Bézier curve is visualised in two ways:

1. **Curve path:** rendered as a white line strip (400 polyline segments) using `BezierCurveRenderer::draw()`.
2. **Railroad ties:** `drawRailroad()` iterates along the arc length in steps of 0.6 units, samples the curve position and tangent at each step, and builds a rotation matrix from the Frenet frame (tangent, up, right). A small, flattened box mesh (`scale(0.1, 0.05, 0.8)`) is placed at each sample point and oriented to follow the local curve direction. This gives the visual impression of a railroad track that bends smoothly along the full loop. The track is redrawn every frame from the currently active curve, so switching curves instantly updates the railroad geometry.

### 5. Camera – Overview Camera & First-Person Camera (Denis)

Three interactive camera modes are available, toggled via keyboard:

| Key | Mode |
|---|---|
| `B` | Free-fly overview camera (mouse look enabled) |
| `F` | First-person view (FPV) locked to the middle carriage |
| Default | Overview camera with right-click mouse look |

The `Camera` class (based on the LearnOpenGL free-look camera) supports `ProcessKeyboard`, `ProcessMouseMovement`, and `ProcessMouseScroll`. In FPV mode (`fpvActive`), the camera position is set every frame to the world position of the middle carriage (index `numCarriages/2 = 3`) with a +1 unit vertical offset, and a `GLFW_CURSOR_DISABLED` raw mouse mode is activated. In overview mode (`cameraMode`), WASD/ZQSD move the camera freely through the scene. The camera orientation towards the current position + tangent happens implicitly because the FPV camera position tracks the carriage.

### 6. Lighting – Local Point Lights with Per-Pixel Shading (Maxim)

Four point lights are placed symmetrically around the track at positions `(+/-8.5, 1, 0)` and `(0, 1, +/-8.5)`. The lighting model is implemented entirely in `basic.frag` as **per-pixel (Phong) shading** in the fragment shader. Each light contributes:

- **Ambient:** low-intensity base illumination (`0.1, 0.1, 0.1`)
- **Diffuse:** Lambertian diffuse term (`dot(N, L)`) scaled by `0.8`
- **Specular:** Specular reflection with shininess = 32, scaled by `1.0`
- **Attenuation:** quadratic falloff `1 / (constant + linear x d + quadratic x d^2)` with coefficients `(1.0, 0.09, 0.032)`, giving a realistic light radius of roughly 30 units.

Each light computes its contribution via `CalcPointLight()` and the results are summed in the fragment shader. Small cube meshes are rendered with the `lampShader` at each light position to make the light sources visible in the scene.

### 7. Convolution – Post-Processing Sharpen Shader (Maxim)

A full-screen **framebuffer object (FBO)** post-processing pipeline is implemented via the `Framebuffer` class. The render loop uses a **two-pass** approach:

1. **First pass:** the entire 3D scene is rendered into `screenFBO` (an off-screen colour + depth buffer).
2. **Second pass:** depth testing is disabled, a full-screen quad is drawn, and `post_sharpen.frag` samples a 3 x 3 neighbourhood of texels from the scene texture and applies a **sharpen convolution kernel**:

```
-0.5  -1.0  -0.5
-1.0   7.0  -1.0
-0.5  -1.0  -0.5
```

The centre weight (7.0) amplifies the original pixel while the negative neighbours subtract surrounding values, effectively enhancing edges and fine detail across the entire rendered frame.

### 10. Interaction – Switching Tracks (Denis)

Two forms of interaction allow the user to jump the train to the alternate Bézier loop:

1. **Keyboard (`C` key):** a rising-edge toggle (`cKeyWasPressed` debounce) flips the `usingSecondCurve` flag and resets the arc-length accumulator to 0, placing the train at the start of the new loop.
2. **Mouse click on the in-world orange button:** `checkButtonInteraction()` projects the 3D button world position `(0, 3, 0)` into screen space using `glm::project()`, computes the 2D pixel distance from the mouse cursor, and triggers the same curve switch if the distance is less than 25 pixels. The button is rendered as a small orange box in the scene.

All interactions:

- Mouse: Look around
- Scroll: Zoom in and out
- WASD/ZQSD: Move camera
- B: Switch between camera mode and free look mode
- F: Switch to first person camera mode on the carts
- Left Mouse Click on orange button: Switch between flat and curved track (in camera mode with cursor on screen)
- C: Switch between flat and curved track
- Space: Move camera up
- Shift: Move camera down
- ESC: Close the application

---

## Features Not Implemented (Due to Time Constraints)

- **8. Post-processing – Bloom / Neon / Halo effect**
- **9. Chroma-keying**

---

## Time Spent

| Date | Who | Work done | Time |
|---|---|---|---|
| 12/04 | Denis | OpenGL lesson 1 exercises (up to §1.2 Rendering) | 3 h |
| 20/04 | Denis | Finished OpenGL exercises; created GitHub repo with initial folder structure | 2 h |

| Date | Who | Work done | Time |
|---|---|---|---|
| 23/04 | Denis | Refactored `Shader.cpp` and `Mesh.cpp`; added `Texture.cpp` (train-carriage.png); added `Camera.cpp` with WASD/ZQSD movement; updated `basic.vert` / `basic.frag` for texture + camera | 3 h |
| 27/04 | Denis | Added Bézier curves (flat xz-plane + heighted variant); added multiple carriages; added Shift/Space camera movement; refactored `main.cpp`; added track-switch via **C**; switched from parameter-based to arc-length-based animation | 3 h |
| 29/04 | Maxim | Completed all OpenGL tutorial exercises in preparation for lighting work | 6 h |
| 07/05 | Denis | Added interaction: **B** toggles free-look / cursor mode; clicking the orange cube switches tracks (mouse picking via `glm::project()`); updated README | 1.5 h |
| 08/05 | Denis | Added first-person view (**F** key); rendered track as a continuous connected line; added source citations for magic-number functions; updated README | 1.5 h |
| 08/05 | Maxim | Added 4 point lights around the track; implemented per-pixel Phong shading in `basic.frag` with quadratic attenuation; added visible lamp cubes | 3 h |

| Date | Who | Work done | Time |
|---|---|---|---|
| 10/05 | Denis | Resolved merge conflicts on main branch | 0.5 h |
| 10/05 | Maxim | Implemented FBO post-processing pipeline with sharpen convolution kernel in `post_sharpen.frag`; attempted bloom on lights (too complex, dropped) | 4 h |
| 13/05 | Denis | Added railroad tie visualisation (`drawRailroad()` with Frenet-frame orientation) | 0.5 h |

**Denis total:** 15 h
**Maxim total:** 13 h
**Grand total: 28 h**